

Project 8: Nonlinear System Identification for a Crop Field Digital-Twin

Please use a new page for each part of a problem. That will help your GSI and grader! See `Spring2026_Project_Instructions.pdf` in files on bcourses for additional instructions.

- **Learning Goal:** Use a genetic algorithm to identify key parameters of a plant growth model from noisy data.
- **Reading:** This project corresponds to Chapter 7 of the course reader, although you will use a slightly different formulation for the model, to be presented in this handout.
 - This assignment involves filling in missing lines in the provided Python template code. Each question will indicate which files and functions should be modified and any necessary plots or console outputs to include with your response. Use the included Jupyter (.ipynb) to organize solutions which require a plot or console output.
 - Required Python packages: `pickle`, `numpy`, `matplotlib`
 - Submit a single, typed .pdf (LaTeX or .ipynb converted to .pdf) that contains all deliverables for this assignment, but not your code. You will submit your code by attaching the modified Python files with your submission.
 - Please use a new page for each part of a problem. That will help your GSI and grader!
- **Introduction** When modeling a physical system, we would always like to validate our model by performing an experiment and collecting measurements of the real system, then checking that the results agree with the model evaluated under the same conditions in the experiment.

In many cases however, there is no way to build a model without first collecting data, since there may be parameters that we can't easily look up or measure directly. For instance, we might be confident in estimating the mass of a car using its CAD assembly, but finding a coefficient relating seasonal rainfall to the water level of a river can only be accomplished through empirical observations, since it is impossible to come up with a deterministic model relating the two a priori (i.e. without observations).

For many kinds of systems, it is more practical to hypothesize a model relating variables of interest with certain trends we expect (i.e. linear, exponential, logarithmic, etc.) and determine the value of its parameters by tuning it with measurements from the real system. We will take this approach to modeling crop growth as we saw in Chapter 7 of the text.



FIGURE 1: Map of cornfield yield measurements. (NC State Dept. of Crop and Soil Sciences)

• Model Formulation

Consider the following model for the growth rate of the heights of plants in a crop-field $\mathbf{h} = (h_1, h_2, \dots, h_n)^T$ with inputs $\mathbf{u} = (w, f, s)^T$, corresponding to water, fertilizer, and solar irradiance (we won't worry about units for any of these quantities). The growth rate $\frac{d\mathbf{h}}{dt}$ is comprised of two terms, one capturing the inputs' effect on the growth rate, the other is an interaction term that slows plant growth due to nearby plants competing for resources.

$$\mathbf{f}(\mathbf{h}(t), \mathbf{u}(t)) = \frac{d\mathbf{h}}{dt} = G(\mathbf{u})\mathbf{h}^\alpha - \mathbf{I}^{\text{int}} \quad (1)$$

Or equivalently for plant i

$$f_i(\mathbf{h}(t), \mathbf{u}(t)) = \frac{dh_i}{dt} = G(\mathbf{u})h_i^\alpha - \sum_{j \neq i} I_{ij}(r_{ij}, h_j) \quad (2)$$

Where the interaction term I_{ij} describes the growth-inhibiting effects of neighboring plants $j \neq i$ at distance r_{ij} from plant i , given by

$$I_{ij} = \lambda h_j \cdot \exp(-r_{ij}/\rho) \cdot \text{Ind}\{r_{ij} \leq \rho\} \quad (3)$$

Where the indicator function takes value 1 when $r_{ij} \leq \rho$, and 0 otherwise, meaning plants at a distance $r_{ij} > \rho$ from plant i do not inhibit its growth.

The scalar growth term

$$G(\mathbf{u}(t)) = G_0(1 + \beta_w w + \beta_f f)s^\gamma \quad (4)$$

includes a base growth rate G_0 and parameters relating irrigation, fertilization, and solar irradiance to the growth rate of the crop.

In terms of free parameters, we have $\alpha, \lambda, \rho, G_0, \beta_w, \beta_f, \gamma$, which we will fit on data gathered from the 'true' system.

• Synthetic Data Generation and Measurement

Instead of using real data, we will generate a growing trajectory using a 'true' set of parameters, plus added noise to simulate measurement error. The height trajectory is given by integrating the growth rate over time and adding a vector of gaussian (normally distributed) noise $\epsilon \sim \mathcal{N}_n(\mathbf{0}, \sigma^2 I_n)$

$$\mathbf{h}(t) = \mathbf{h}(t = t_0) + \int_{t_0}^t \mathbf{f}(\mathbf{h}(t), \mathbf{u}(t))dt + \epsilon \quad (5)$$

Which we form a discrete approximation of with explicit numerical integration, sampling a new noise vector ϵ_k at each timestep.

$$\mathbf{h}_{k+1} \approx \mathbf{h}_k + \Delta t \mathbf{f}(\mathbf{h}_k, \mathbf{u}_k) + \epsilon_{k+1} \quad (6)$$

For simplicity we will take both the integration timestep and observation timestep to be the same, one day. The initial heights $\mathbf{h}_{k=0}$ are randomly sampled from a small interval, and you will be provided a function that drives the inputs in $\mathbf{u}_k = (w_k, f_k, s_k)$, which follow a burst, step, and sinusoidal pattern respectively.

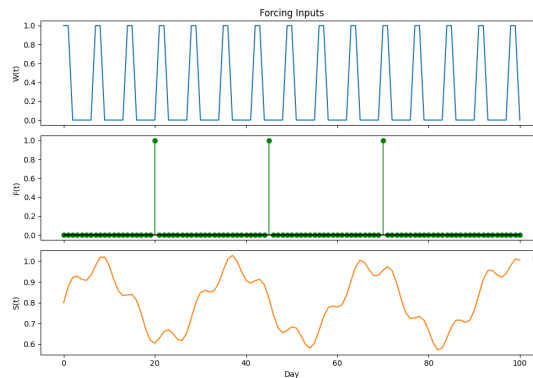


FIGURE 2: Irrigation, fertilizer, and irradiance inputs over the growing season

- **Objective** Estimate the model parameters $\Lambda = \{\alpha, \lambda, \rho, G_0, \beta_w, \beta_f, \gamma\}$ by minimizing the mean-squared error (MSE) between our model and the data using the genetic algorithm, i.e.

$$\Pi(\Lambda) = \frac{1}{n_k} \sum_{k=0}^{n_k-1} \frac{1}{n_p} \|\mathbf{h}_k^{\text{pred}}(\Lambda) - \mathbf{h}_k^{\text{obs}}\|^2$$

where n_p is the number of plants and n_k is the number of observations.

- **Deliverables** (100 points)

For all parts, use the default parameters for the simulation, data generation, and genetic algorithm Python functions unless otherwise specified (see part 3(c)).

1. Synthetic Data Generation

- (10 Points) The included function `data_generation.py` calls the simulation function that numerically integrates the growth equation using the approximation in (6). It generates a training data set and a testing data set, which come from different growing seasons with different inputs $\mathbf{u}_{\text{train}}, \mathbf{u}_{\text{test}}$. Why is it important to evaluate our model, (say, in terms of mean squared error), on an unseen test set?
- (25 Points) Following the instructions in Part 1 of the included notebook `main.ipynb`, complete the model and include an image of the generated train and test trajectories output by the notebook cell.

2. Genetic Algorithm

- (15 Points) We will be using the Genetic Algorithm to search for the parameters that fit our data. We have chosen a model that has 7 free parameters, which are shared by all plants. Suppose we decided to fit a set of these parameters individually for each plant, leaving us with $7 \text{ params} \cdot 36 \text{ plants} = 252$ free parameters. Describe (just qualitatively, as we can't prove it numerically) how the increased number of parameters will impact our training error (i.e. lowest cost achieved with the GA). Will our model overfit or underfit the data? See [this resource](#) for a concise introduction to the concept of overfitting, an essential concept in Machine Learning.
- (10 Points) Complete the MSE function as outlined in Part 3 of `main.ipynb`. How does this differ from the cost function presented in Chapter 7 of the textbook?

3. Parameter Recovery

- (10 Points) Run the cells in Parts 3-4 of the included notebook to run the GA, get a summary table for the parameter recovery, and see plots of our estimated vs. true trajectories for both the train and test season. Include the parameter table and plot with your submission. Does the model generalize well when tested on the left-out set?
- (10 points) Which parameters are recovered accurately? Which are harder to identify, and why? Does this have a big impact on the predictive capability of our model?
- (20 points) Generate new data by increasing the `noise_sigma` parameter of `generate_dataset` and report your parameter recovery table after running the GA on this new data. How does increasing the noise level of our model impact our ability to recover parameters accurately?